



АКАДЕМИЯ
КОДЕБАЙ

ПРОФЕССИЯ ПЕНТЕСТЕР

Друзья, всех приветствую! В этом уроке мы разберем работу с функциями.

Функции в Bash позволяют сделать код более структурированным, читаемым и многократно используемым. Вместо повторения одинаковых команд, можно вынести их в отдельную функцию и вызывать её по мере необходимости. Это не только ускоряет написание скриптов, но и упрощает их сопровождение: если нужно внести изменения, их достаточно сделать в одном месте. Функции помогают разбивать сложные задачи на небольшие, логически связанные шаги, что делает код более гибким и понятным.

С точки зрения сценариев Bash, мы можем рассматривать функцию как сценарий внутри сценария, что полезно, когда нам нужно выполнить один и тот же код несколько раз. Вместо того чтобы переписывать один и тот же кусок кода снова и снова, мы просто пишем его один раз как функцию, а затем вызываем эту функцию по мере необходимости.

Другими словами, функция – это подпрограмма или блок кода, который реализует набор операций. Функции могут быть записаны в двух различных форматах. Первый формат более характерен для сценариев Bash:

```
function ИМЯ_ФУНКЦИИ {  
    КОМАНДЫ ...  
}
```

Второй формат более знаком C-программистам:

```
ИМЯ_ФУНКЦИИ () {  
    КОМАНДЫ ...  
}
```

Эти форматы функционально идентичны и являются вопросом личных предпочтений. Давайте рассмотрим простой пример:

```
kali@kali:~$ cat > ./func.sh  
#!/bin/bash  
print_me () {  
    echo "HELLO WORLD!"
```

```
}  
print_me  
kali@kali:~$ chmod +x ./func.sh  
kali@kali:~$ ./func.sh  
HELLO WORLD!
```

Функции также могут принимать аргументы:

```
kali@kali:~$ cat > ./funcarg.sh  
#!/bin/bash  
pass_arg() {  
echo "Случайное число: $1"  
}  
pass_arg $RANDOM  
kali@kali:~$ chmod +x ./funcarg.sh  
kali@kali:~$ ./funcarg.sh  
Случайное число: 25207
```

В данном случае мы передали случайное число `$RANDOM` в функцию, которая выводит его как первый аргумент функции - `$1`. В Bash аргументы передаются через специальные переменные: первый аргумент хранится в переменной `$1`, второй — в `$2` и так далее. Обратите внимание, что определение функции (`pass_arg()`) содержит круглые скобки. В других языках программирования, таких как C, они содержали бы ожидаемые аргументы, но в Bash круглые скобки служат только для украшения. Они никогда не используются. Также обратите внимание, что мы должны определить функцию в сценарии до ее вызова. С точки зрения логики, мы не можем вызвать то, что мы не определили.

Чтобы получить список всех аргументов, переданных функции в BASH, можно использовать специальную переменную `$@`. Она содержит все аргументы, переданные функции или скрипту. Каждый аргумент можно обработать по отдельности, например, с помощью цикла `for`.

```
list_arguments() {  
    echo "Переданные аргументы:"  
    for arg in "$@"; do  
        echo "$arg"  
    done  
}
```

Функция выше содержит цикл, который перебирает все переданные в функцию аргументы и выводит их результат на экран. Таким образом, мы увидим все переданные аргументы, если укажем их после имени функции в момент вызова:

```
list_arguments 1 2 3 4 5  
Переданные аргументы:  
1  
2  
3  
4  
5
```

Так же я рекомендую вам использовать описательное имя функции, которое описывает ее назначение, то есть важно придумывать такое имя для функции, которое будет понятно вам же при чтении этого кода. Старайтесь добавлять в имя функции какой-то глагол – так будет проще отличить этот объект в коде от переменных.

Помимо передачи аргументов в функции Bash, мы можем возвращать значения из функций Bash. Функции Bash на самом деле не позволяют возвращать произвольное значение в традиционном смысле. Вместо этого функция Bash может возвращать статус выхода (нулевой для успеха, ненулевой для неудачи) или какое-то другое произвольное значение, к которому мы можем позже получить доступ из глобальной переменной \$?.

Рассмотрим простой пример, который возвращает случайное число в глобальную переменную \$?:

```
kali@kali:~$ cat > ./funcrvalue.sh
#!/bin/bash
return_me() {
return $RANDOM
}
return_me
echo "Функция вернула значение $?"
kali@kali:~$ chmod +x ./funcrvalue.sh
kali@kali:~$ ./funcrvalue.sh
Функция вернула значение 587
kali@kali:~$ ./funcrvalue.sh
Функция вернула значение 143
```

Обратите внимание, что случайное число возвращается каждый раз при запуске сценария, потому что мы вернули специальную глобальную переменную \$RANDOM (в \$?). Если бы мы использовали оператор return без аргумента \$RANDOM, то вместо него возвращалось бы состояние выхода из функции (в данном случае 0).

Теперь, когда у нас есть базовое понимание переменных и функций, мы можем углубиться и обсудить область видимости переменной. Область видимости переменной – это просто контекст, в котором она имеет значение. По умолчанию переменная имеет глобальную область видимости, то есть к ней можно получить доступ во всем сценарии. В отличие от этого, локальная переменная может быть видна только в пределах функции или блока кода, в котором она определена. Мы можем глобальной переменной придать локальный контекст, предваряя объявление ключевым словом local. Общий синтаксис таков:

```
local ИМЯ_ПЕРЕМЕННОЙ=ЗНАЧЕНИЕ
```

Давайте посмотрим, как локальные и глобальные переменные работают на практике на простом примере:

```
kali@kali:~$ cat > ./varscope.sh
#!/bin/bash
name1="Миша"
name2="Катя"
name_change() {
local name1="Ваня"
echo "Внутри этой функции name1 - это $name1, а name2 - это $name2"
name2="Олег"
}
echo "До вызова функции name1 - это $name1, а name2 - это $name2"
name_change
echo "После вызова функции name1 - это $name1, а name2 - это $name2"
kali@kali:~$ chmod +x ./varscope.sh
kali@kali:~$ ./varscope.sh
До вызова функции name1 - это Миша, а name2 - это Катя
Внутри этой функции name1 - это Ваня, а name2 - это Катя
После вызова функции name1 - это Миша, а name2 - это Олег
```

Давайте выделим несколько ключевых моментов. Во-первых, обратите внимание, что мы объявили две глобальные переменные, установив name1 в Миша и name2 в Катя. Затем мы определили функцию и внутри этой функции объявили локальную переменную name1, задав для нее значение Ваня. Далее, мы устанавливаем name2 в Олег, и поскольку мы не использовали ключевое слово local, мы изменяем глобальную переменную, и присваивание сохраняется как внутри, так и вне функции. Исходя из этого примера, можно сказать, что:

1. Изменение значения локальной переменной с тем же именем, что и у глобальной, не влияет на ее глобальное значение.

2. Изменение значения глобальной переменной внутри функции (без объявления локальной переменной с тем же именем) - повлияет на ее глобальное значение.

Знание функций в Bash — это важный шаг на пути к эффективному использованию командной строки. Они позволяют создавать более мощные и универсальные скрипты, которые легко адаптировать и поддерживать. Функции упрощают работу над проектами, помогают избегать ошибок и обеспечивают лучшую читаемость кода. Важно продолжать практиковаться, создавая свои функции для автоматизации повседневных задач, что сделает вашу работу с Bash более продуктивной и удобной.